RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world®

VI semester

Experiential Learning Report on SDG – 9

SCOUT : Supply Chain Outage Understanding & Tracker

Students

Sl. No	USN	Name
1	1RV23IS017	Ankit Pathak
2	1RV24AI406	L Moryakantha
3	1RV23AI091	Shashank K
4	1RV23IS015	Anirudh C
5	1RV23CS296	Tulya Reddy

Mentor	Chief Mentor
Prof. Rajatha	**Dr. K N Subramanya**
Assistant Professor	*Professor & Principal*
Department Name	*RVCE*

Even 2025–26

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

Certified that the **Experiential Learning on SDG 9** project work titled **"SCOUT - Supply Chain Outage Understanding Tracker"** is carried out by **Ankit Pathak (1RV23IS017)**, **L Moryakantha (1RV24AI406)**, **Shashank K (1RV23AI091)**, **Anirudh C (1RV23IS015)**, and **Tulya Reddy Y (1RV23CS296)**, who are bonafide students of RV College of Engineering, Bengaluru, in fulfillment of the curriculum requirement of 6th Semester Experiential Learning during the academic year **2025–2026**. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of Experiential Learning work prescribed by the institution.

Faculty In-charge	**Head of the Department**	**Principal**
Prof. Rajatha	Dr. Shanta Rangaswamy	Dr. K N Subramanya

ACKNOWLEDGEMENTS

We are deeply indebted to our Mentor, **Prof. Rajatha**, Assistant Professor of Department of Computer Science and Engineering, RVCE, for the wholehearted support, suggestions, and invaluable advice throughout our 6th Semester Experiential Learning project work titled **"SCOUT - Supply Chain Outage Understanding Tracker"** and for her guidance in the preparation of this report.

We also express our gratitude to our panel members and evaluators for their valuable comments and suggestions during the phase evaluations, which significantly contributed to aligning our project with Sustainable Development Goal 9 (SDG 9).

Our sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE, for the constant support, facilities provided, and encouragement.

We express our sincere gratitude to our beloved Vice Principal, **Dr. K.S. Geetha**, RVCE, and Principal, **Dr. K N Subramanya**, RVCE, for their appreciation and visionary support towards Experiential Learning and interdisciplinary projects.

We are deeply grateful to the Dean Academics, RVCE, and the experiential learning coordinators for their continuous support in the successful execution of this multidisciplinary team project.

We would also like to thank **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized LaTeX template which made our report writing easy and interesting.

We thank all the teaching and technical staff of RVCE for their continuous help and guidance.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support and encouragement throughout the project work.

Contents

1 Abstract

Modern supply chain organizations operating in dynamic environments require systems capable of ingesting live events, preserving lineage, and reasoning over complex relationships. Traditional data pipelines frequently stop at basic cleaning or dashboarding, failing to provide a unified platform capable of converting raw, heterogeneous telemetry into governed, graph-aware, and AI-assisted operational decisions. To address this gap, this project introduces SCOUT (Supply Chain Outage Understanding & Tracker), a Palantir-tier intelligence data operating system. SCOUT is designed to connect live data ingestion, real-time state management, graph ontology, optimization, scenario simulation, and autonomous machine-learning retraining into a single integrated decision-support environment.

The system's architecture utilizes a multi-layered distributed platform built on containerized infrastructure. Heterogeneous data from six primary sources, including GDELT, NewsAPI, and ACLED, is ingested continuously through a rate-limit-aware connector framework that applies SHA-256 content hashing for deduplication and provenance preservation. This raw data undergoes schema-agnostic normalisation before entering an independently scalable, Celery-based natural language processing pipeline. Within this pipeline, fine-tuned spaCy transformer models extract named entities, while DistilBERT sequence classifiers categorize supply-chain disruption types. These extracted entities are subsequently linked to controlled vocabularies, such as UN/LOCODE and ISO 3166-1, to ensure standardized graph mapping and accurate relationship reasoning.

To prevent alert fatigue and generate actionable intelligence, SCOUT employs a weighted composite risk scoring engine. This engine calculates operational risk by evaluating event severity, exponential recency decay, source credibility, and supplier relevance, while applying criticality multipliers for sole-source versus replaceable suppliers. The aggregated scores dynamically map to specific alert tiers, driving visual dashboard updates and automated notifications. Furthermore, the platform integrates optimization engines and scenario simulation capabilities, allowing operators to construct and execute visual workflows through a Directed Acyclic Graph (DAG) interface using React Flow.

Performance evaluations demonstrate that SCOUT significantly outperforms traditional static ETL baselines. The system achieved a maximum ingestion throughput of 10.4k events per second with a mean latency of 85 milliseconds. Logic reconfiguration times were reduced from 1200 seconds to 0.5 seconds, while AI linking accuracy improved to 91%. The use of background thread-pool offloading successfully mitigated main-thread blocking during large graph exports from Neo4j, ensuring real-time responsiveness. Ultimately, SCOUT establishes a robust, reproducible methodology for transforming fragmented, unstructured signals into a governed, scalable, and operationally mature intelligence network.

2 Introduction

SCOUT is a Palantir-tier intelligence data operating system designed to connect live data ingestion, real-time state management, graph ontology, optimization, scenario simulation, governance, and autonomous machine-learning retraining. The term "operating system" in this project refers to a data and intelligence operating system: a platform that coordinates many data services and AI modules into one integrated decision-support environment.

In today's highly volatile global landscape, supply chain networks are increasingly vulnerable to cascading disruptions, ranging from geopolitical conflicts and extreme weather events to sudden macroeconomic shifts. Traditional data pipelines frequently fall short in these dynamic environments, as they typically conclude their operations at the data cleaning or dashboarding stages. This leaves a critical gap in organizational readiness: a profound lack of a unified platform capable of synthesizing raw, heterogeneous telemetry into governed, actionable, and graph-aware operational decisions. Organizations require more than static alerts; they need systems that actively reason over multi-tier supplier relationships, trace the provenance of every data point, and continuously learn from ongoing events.

To bridge this operational gap, SCOUT is engineered to process a diverse array of unstructured, semi-structured, and structured external signals. Operational intelligence rarely arrives in a standardized format; a disruption may first manifest as negative sentiment in a news article, subsequently appear as a registered conflict event, and eventually reflect as a spike in freight indices or commodity prices. By continuously ingesting data from representative sources such as GDELT event streams, NewsAPI feeds, ACLED conflict registers, and macroeconomic indicators, SCOUT establishes a multi-dimensional foundation of the global operational landscape. This multi-source fusion ensures that critical warning signs are captured and deduplicated regardless of their initial reporting format.

The project has evolved through fifteen phases, beginning with basic data pipeline functionality and progressing toward autonomous active-learning behavior. Its architecture uses containerized infrastructure services, a Python FastAPI backend, background ingestion daemons, graph databases, caching layers, data-lake archival, and a React Flow visual interface for composing workflows. Modern intelligence platforms are built around the idea that raw data must be converted into operational knowledge. Data pipelines ingest and transform events, stream-processing systems route data at scale, graph databases represent relationships, caches support low-latency world-state queries, and machine-learning models infer risk or optimize action. SCOUT follows this design direction by integrating PostgreSQL, MQTT, Neo4j, Redis, Redpanda Kafka, Parquet archival, FastAPI, and React Flow.

Ultimately, SCOUT goes beyond standard data aggregation through a scalable Natural Language Processing (NLP) pipeline and weighted risk scoring engine. Transformer-based models extract entities, classify disruption types, and map events to controlled vocabularies and geographic taxonomies. The enriched records are evaluated using supplier criticality, recency, and source credibility to produce meaningful alert tiers, helping decision-makers proactively mitigate complex supply-chain outages.

3 Problem Definition

3.1 Problem Statement

Organizations operating in dynamic supply-chain environments need systems that can ingest live events, preserve lineage, reason over relationships, optimize responses, and learn from feedback. Traditional pipelines usually stop at cleaning, storage, or dashboarding, leaving a gap between raw telemetry and actionable operational decisions.

SCOUT addresses this gap by designing an integrated data operating system that converts heterogeneous telemetry into governed, graph-aware, and AI-assisted intelligence. The platform supports live ingestion, deduplication, real-time and graph-based state management, visual workflow execution, AI-assisted optimization, and feedback-driven model improvement.

3.2 Background Information: Literature Review

The literature related to SCOUT can be grouped into five themes: disruption propagation, misinformation filtering, event identification from news, sentiment-based disruption detection, and sustainable inventory optimization. Together, these works show that modern supply-chain risk systems must combine external signal ingestion, NLP, event classification, risk scoring, provenance, and optimization rather than functioning as static dashboards.

[1] Herold & Marzantowicz (2023) — Disruption Propagation

Herold and Marzantowicz explain how supply-chain disruptions create ripple effects across suppliers, logistics providers, regulators, buyers, and financial stakeholders. Their institutional-complexity perspective shows that disruptions are shaped by organizational pressures and actor-specific interpretations. For SCOUT, this motivates the use of a graph-aware operating system instead of a simple alert list. The limitation is that the work remains conceptual; SCOUT operationalizes this idea through ingestion connectors, Neo4j ontology mapping, Redis world-state caching, and automated risk computation.

[2] Akhtar et al. (2023) — Misinformation Filtering

Akhtar et al. show how AI and machine learning can filter fake news and disinformation before they affect supply-chain decisions. This is important because SCOUT relies on open-source feeds such as news articles, public event streams, and external indicators. The paper supports SCOUT's emphasis on validation, deduplication, provenance, and confidence scoring in the ingestion layer. However, it does not fully address multi-source event fusion or downstream risk scoring, which SCOUT adds through normalization, entity linking, and risk-tier generation.

[3] Shahsavari et al. (2024) — LLM-Based Event Identification

Shahsavari, Hussain, Saberi, and Sharma introduce an LLM-based method for identifying supply-chain risk events from news. Their approach moves beyond keyword search by extracting event

names, causes, locations, and related phrases. This aligns with SCOUT's NLP pipeline, where sources such as NewsAPI and GDELT are converted into structured event records. The limitation is that the work focuses mainly on news-based event extraction; SCOUT extends it with graph relationships, weighted risk scoring, and decision-support workflows.

[4] Vishnuthilak et al. (2024) — Sentiment-Based Risk Detection

Vishnuthilak et al. use sentiment analysis to detect disruptive supply-chain events by risk type, geography, frequency, and headline sentiment. Their work supports SCOUT's use of textual sentiment as one factor in a broader composite risk engine that also considers source reliability, entity importance, geography, and market indicators. The gap is that sentiment analysis alone does not capture multi-tier supplier propagation; SCOUT links sentiment-enriched records to suppliers, regions, commodities, and graph nodes before generating alerts.

[5] Aiello et al. (2025) — Sustainable Inventory Optimization

Aiello et al. present a sustainable closed-loop inventory model using reuse, recovery, recycling, and waste-reduction logic. Although it is not a real-time disruption-monitoring system, it shows why risk intelligence should ultimately support operational optimization. For SCOUT, this informs scenario simulation and response planning, where detected risks can guide rerouting, inventory reallocation, or more sustainable mitigation decisions.

Summary

Overall, the reviewed literature provides useful foundations but does not offer a complete live intelligence platform. SCOUT combines these contributions into one architecture for ingestion, deduplication, provenance, NLP extraction, graph ontology, risk scoring, scenario simulation, governance, and active learning (see Figure 12.8).

4 Objectives

4.1 Primary Objectives

1. To analyze the multi-layer infrastructure and application architecture of the SCOUT intelligence data operating system, specifically detailing its distributed big-data lakehouse model across Bronze, Silver, and Gold tiers.
2. To document the fifteen functional phases of system maturity, migrating from basic telemetry ingestion pipelines to autonomous Graph Neural Network (GNN)-driven active-learning loops using PyTorch.
3. To demonstrate how live, heterogeneous telemetry can be transformed into predictive, graph-aware operational intelligence, specifically addressing the anticipation gap and reactive limitations currently faced by Small and Medium-sized Enterprises (SMEs).
4. To present the complete operational lifecycle, including startup orchestration, distributed data extraction, cryptographic governance stamping, scenario simulation, and the evaluation workflow for the platform.

4.2 Secondary Objectives

1. To study the integration and interaction of a massive-scale enterprise technology stack, including how Databricks, Apache Spark, Apache Flink, MinIO, and Apache Iceberg operate alongside PostgreSQL, MQTT, Neo4j, Redis, FastAPI, and React Flow.
2. To evaluate user-centric personalization mechanisms, specifically studying the implementation of supplier profile onboarding, custom severity filtering, and targeted delivery networks such as real-time geospatial heatmaps and automated email digests.
3. To thoroughly understand the mechanisms of data-lineage preservation, low-latency world-state caching, and continuous multi-tier relationship reasoning within the graph ontology.
4. To contrast SCOUT's predictive, NLP-enriched capabilities against traditional Enterprise Resource Planning (ERP) and standard supply-chain visibility platforms, identifying future avenues for platform expansion.

5 Methodology

5.1 Methodological Overview

The methodology adopted for SCOUT follows a layered data-operating-system model in which heterogeneous external information is first ingested, then converted into a common canonical representation, then enriched through natural-language and vocabulary-linking modules, and finally converted into operational risk scores and alerts. The three method-ology diagrams used for this section represent the major stages of this pipeline. Figure 1 describes the multi-source heterogeneous ingestion and schema-agnostic normalisation layer. Figure 2 describes the NLP-driven supply-chain signal extraction and classifica-tion pipeline. Figure 3 describes the weighted composite risk scoring engine that converts enriched records into actionable alert tiers. The overall method is therefore not limited to one data-processing script. It is a complete workflow that starts with raw signals from external data sources and ends with structured, queryable, and prioritized intelligence records. This is suitable for an intelligence data operating system because such systems must handle incomplete, noisy, delayed, dupli-cated, and differently formatted data. The methodology deliberately separates ingestion,

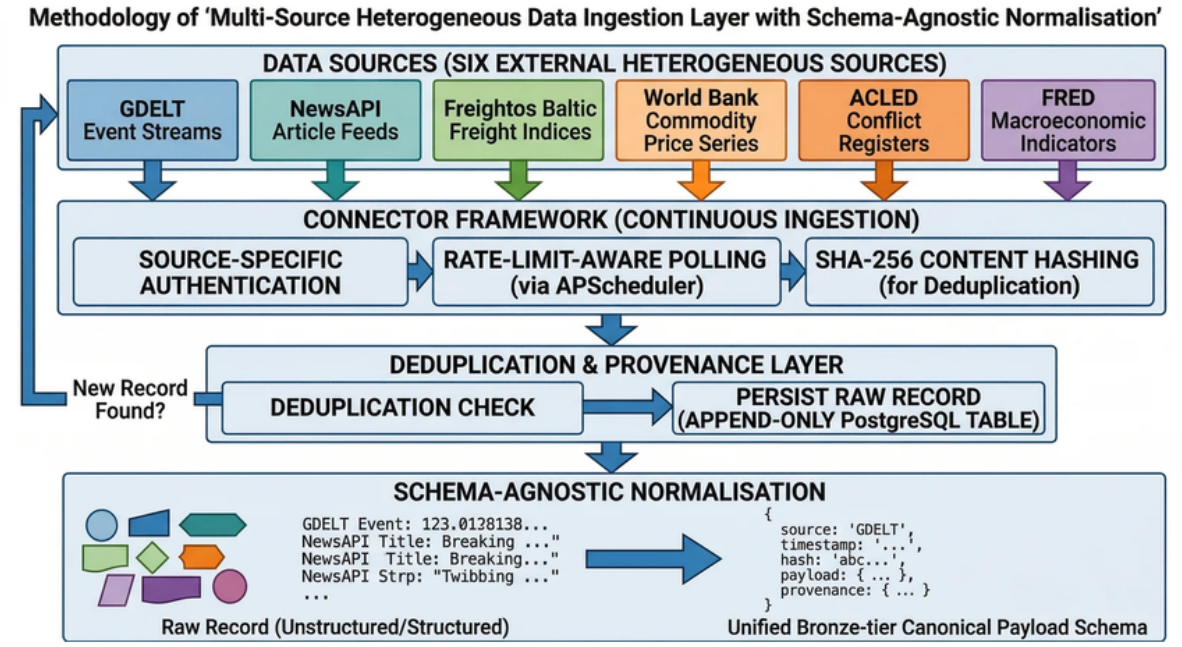

Figure 5.1: Multi-source heterogeneous data ingestion layer with schema-agnostic normalisation

normalisation, enrichment, classification, risk scoring, and alerting so that each stage can be validated independently and improved without disturbing the full system. The proposed pipeline can be summarized as follows: External Sources â†’ Connector Framework â†’ Deduplication and Provenance â†’ Schema-Agnostic Normalisation â†’ NLP Entity Extraction â†’ Signal Classifi-cation â†’ Vocabulary Linking â†’ Gold-Tier Event Records â†’ Composite Risk Scoring â†’ Alert Delivery This methodology supports the objective of SCOUT because it converts free-form oper-ational data

into governed, structured, and decision-ready information. The design also supports traceability. At every major step, records preserve source identity, timestamps, hashes, provenance metadata, and processing outputs. This is important because oper-ational intelligence is valuable only when its origin and transformation history can be inspected.

5.2 Stage 1: Heterogeneous Data Source Selection

The first methodological stage is the selection of heterogeneous sources. The ingestion layer is designed around six representative external sources: GDELT event streams, News- API article feeds, Freightos Baltic freight indices, World Bank commodity price series, ACLED conflict registers, and FRED macroeconomic indicators. These sources were se-lected because they represent different classes of real-world data. Some are event streams, some are article feeds, some are index values, some are commodity time series, some are conflict records, and some are macroeconomic indicators. This diversity is essential for testing whether the platform can operate beyond a single fixed schema. The use of heterogeneous sources also reflects the real nature of supply-chain and operational-risk analysis. A disruption signal may appear first as a news article, later as a conflict event, then as a freight-rate change, and finally as a macroeconomic indicator. A method-ology that uses only one source type would fail to capture these cross-domain relation-ships. SCOUT therefore treats source diversity as a core design requirement rather than as an optional extension. Each source contributes a different kind of evidence. GDELT and NewsAPI provide text-heavy, fast-moving reports about geopolitical, economic, and logistics events. Freightos Baltic data provides freight-market indicators that can reveal stress in shipping routes. World Bank commodity information provides material-price context. ACLED contributes structured conflict and political-violence evidence. FRED contributes macroeconomic background signals such as inflation, interest-rate, employment, and production indica-tors. Together, these sources create a multi-dimensional view of operational risk.

5.3 Stage 2: Continuous Connector Framework

After source selection, the next stage is continuous ingestion through a connector frame-work. The framework performs source-specific authentication, rate-limit-aware polling, and SHA-256 content hashing. Source-specific authentication is necessary because each external service can use a different access mechanism. Some sources may require API keys, some may use public endpoints, and some may impose query-specific limits. The connector layer hides these differences behind a common ingestion interface. Rate-limit-aware polling is used to make the ingestion system reliable and respectful of external service limits. Instead of repeatedly calling APIs without control, SCOUT schedules polling based on each source's update frequency and allowed request rate. This prevents accidental API blocking and also reduces unnecessary backend load. A scheduler-based approach makes the ingestion process more predictable because each connector can be configured according to its data velocity and importance. The third function of the connector framework is SHA-256 content hashing. Every fetched record is converted into a stable hash based on its meaningful content. This hash acts as a compact fingerprint of the record. If the same article, event, or data point appears again, the system can detect it without comparing every field manually. Hashing also supports

provenance because a stored record can later be linked to the exact content that entered the system. The connector framework is designed to operate continuously. This means ingestion is not treated as a one-time file upload but as a background operational process. As new events appear, they move through the same validation, deduplication, and normalisation stages. This continuous design is important for SCOUT because the platform aims to support real-time and near-real-time decision support.

5.4 Stage 3: Deduplication and Provenance Preservation

The next stage is the deduplication and provenance layer. Once a connector retrieves a record and computes its hash, the system performs a deduplication check. If the hash already exists, the record can be ignored, updated, or linked as a repeated observation depending on the use case. If it is a new record, it is persisted as an append-only raw record in PostgreSQL. The append-only approach is methodologically important. Instead of overwriting pre-vious records, the system preserves the raw incoming version. This ensures that later transformations can be audited. If an NLP model produces an incorrect classification, the original input is still available for review and reprocessing. Similarly, if a source corrects a previous entry, both versions can be retained with timestamps and provenance information. Provenance preservation includes storing the source name, ingestion timestamp, source URL or endpoint metadata where available, content hash, raw payload, and processing status. These fields allow the system to answer questions such as where a record came from, when it was collected, whether it was duplicated, and which downstream processes used it. This is especially important in a research setting because results must be repro-ducible and explainable. In SCOUT, deduplication also improves the quality of later NLP and risk-scoring out-puts. Without deduplication, the same article or event could be counted multiple times, inflating risk scores. By removing or linking duplicates before scoring, the system reduces false escalation and keeps alert generation more meaningful.

5.5 Stage 4: Schema-Agnostic Normalisation

Raw source records rarely share the same schema. For example, a GDELT event may contain event codes, actor fields, locations, and tone values. A NewsAPI article may contain a headline, author, description, URL, and publication time. A freight index may contain route, price, and date fields. A commodity price record may contain commodity name, unit, price, and reporting period. A conflict register may contain actor, event type, latitude, longitude, and fatalities. A macroeconomic indicator may contain series code, value, date, and country. The schema-agnostic normalisation stage solves this problem by converting every raw record into a bronze-tier canonical payload. The canonical payload contains a stable set of top-level fields such as source, timestamp, hash, payload, and provenance. The detailed original data is retained inside the payload field, while standard metadata is exposed consistently for downstream modules. This gives the system both flexibility and structure: it does not lose the original schema, but it still provides a common wrapper for processing. This stage is called schema-agnostic because it does not assume that every source has the same columns. Instead, it accepts structured, semi-structured, and unstructured data and

maps them into a shared representation. This enables the same backend pipeline to process event streams, text articles, numerical indicators, and registry records. Normalisation also establishes the bronze-tier layer of the data lifecycle. The bronze tier contains raw or lightly processed canonical records. Later stages can produce silver-tier enriched records and gold-tier decision records. This tiered methodology makes the data lifecycle easier to reason about. Bronze records preserve source truth, enriched records add extracted meaning, and gold records represent validated intelligence outputs.

5.6 Stage 5: NLP-Based Signal Extraction

After normalisation, unstructured and semi-structured text records move into the NLP-driven signal extraction pipeline shown in Figure 2. This stage focuses mainly on news text and article-like records because these sources contain valuable operational signals in natural language. The goal is to convert free-form text into structured annotations that

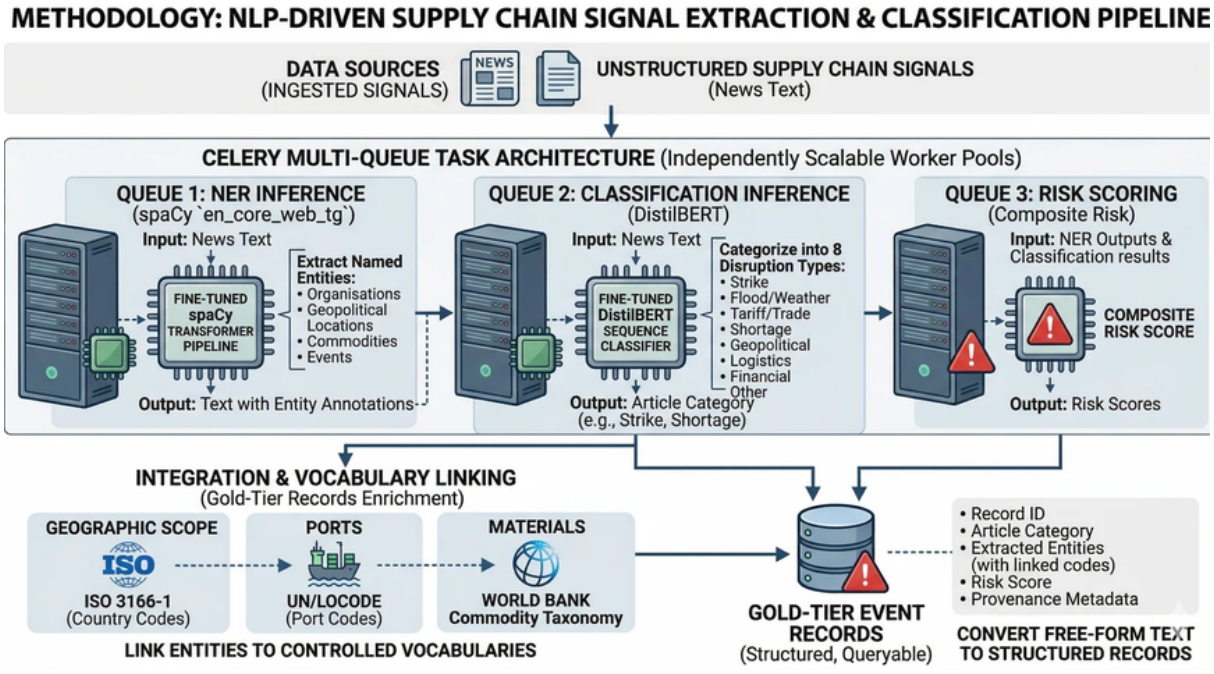

Figure 5.2: NLP-driven supply-chain signal extraction and classification pipeline

downstream systems can query. The first NLP queue performs named entity recognition using a fine-tuned spaCy trans-former pipeline. Named entity recognition identifies important entities such as organi-zations, geopolitical locations, commodities, and events. For example, an article about port congestion may mention a country, a port, a shipping company, and a commodity. These mentions are not useful to a database unless they are extracted into fields. NER therefore acts as a bridge between natural language and structured intelligence. The NER stage produces text with entity annotations. These annotations become part of the enriched record. The methodology uses a queue-based design because NLP inference can be computationally heavier than simple ingestion.

By assigning NER to its own queue, the system can scale workers independently. If the number of incoming articles increases, additional NER workers can be started without changing the connector or scoring logic. This queue-based architecture also supports fault isolation. If the NER model fails on a malformed article, the rest of the ingestion pipeline can continue. Failed records can be marked for retry or manual inspection. This improves robustness compared with a single monolithic pipeline where one error could stop all processing.

5.7 Stage 6: Classification of Supply-Chain Disruption Types

The second NLP queue performs classification inference using a fine-tuned DistilBERT sequence classifier. The classifier receives article text and categorizes it into disruption types such as strike, flood/weather, tariff/trade, shortage, geopolitical, logistics, financial, or other. This classification step is important because a risk platform must know not only that an article exists, but also what kind of operational problem it describes. DistilBERT is suitable for this role because it can represent sentence-level meaning while remaining lighter than larger transformer models. In the SCOUT methodology, the classifier output becomes an article category or disruption label. This label is later used by the risk-scoring engine as one of the signal components. For example, a strike near a port may receive a different operational interpretation from a general financial report or a weather event. Classification also improves dashboard filtering and user interpretation. Operators can filter records by disruption type and focus on the categories relevant to their workflow. A logistics team may focus on port and route disruptions, while a procurement team may focus on commodity shortages and price shocks. By generating consistent categories, the system turns scattered articles into organized intelligence. The queue-based classification design again allows independent scaling. Text classification workers can be increased during high news volume, while NER workers and risk-scoring workers can scale according to their own workloads. This supports the data-operating-system goal of modular, independently scalable services.

5.8 Stage 7: Integration and Controlled Vocabulary Linking

After entity extraction and classification, the methodology links extracted entities to controlled vocabularies. The diagram identifies three important vocabulary families: ge-ographic scope, ports, and materials. Geographic scope is linked through ISO 3166-1 country codes. Ports are linked through UN/LOCODE port codes. Materials are linked through a commodity taxonomy such as the World Bank commodity taxonomy. Vocabulary linking solves a major practical problem: the same entity can appear in many forms. For example, a country may be written as “United States,” “USA,” or “U.S.” A port may be referred to by its full name, local spelling, or abbreviation. A material may appear under a commercial name or a broader commodity category. Without controlled vocabularies, the graph layer and risk engine would treat these variations as different entities. By linking entities to standardized identifiers, SCOUT creates consistent records that can be queried reliably. A user can search for all records connected to a country code, port code, or commodity group even if the original articles used different wording. This makes the system more useful for research and operational analysis because it reduces ambiguity. The output of this stage

is a gold-tier event record. A gold-tier record contains the record ID, article category, extracted entities with linked codes, risk score, and provenance metadata. It is called gold-tier because it is no longer just raw data; it is enriched, structured, and ready for querying, dashboarding, graph mapping, and decision support.

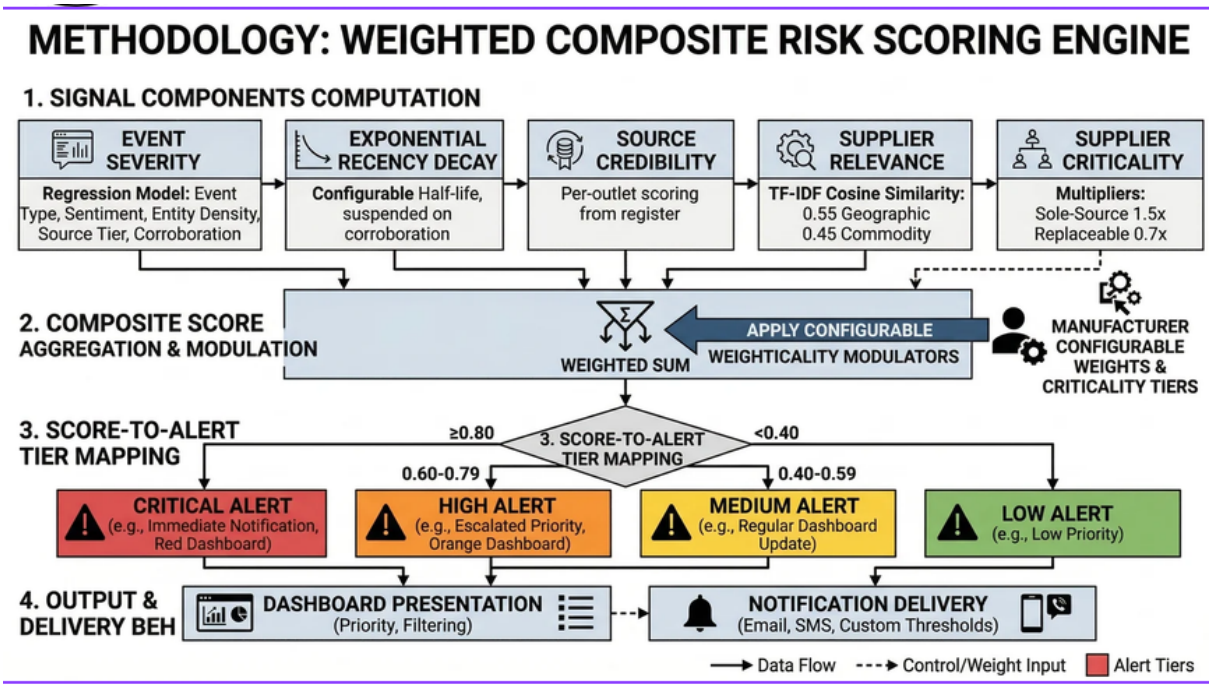

Figure 5.3: Weighted composite risk scoring engine

5.9 Stage 8: Weighted Composite Risk Scoring

The final analytical stage is the weighted composite risk scoring engine operational risk scores. The scoring methodology is based on multiple signal components rather than a single binary rule. This is important because real-world risk is rarely determined by one factor. A highly relevant event from a weak source may not deserve the same priority as a corroborated event from a credible source affecting a critical supplier. The first component is event severity. Severity can be estimated from event type, sen-timent, entity density, source tier, and corroboration. A violent conflict event, a port closure, or a major shortage would normally receive a higher severity value than a rou-tine market update. Sentiment can indicate whether the language is negative or urgent. Entity density can indicate whether the article contains many relevant operational en-tities. Corroboration can increase severity confidence if multiple independent sources report similar facts. The second component is exponential recency decay. Operational intelligence loses value as it becomes older. A disruption reported today is usually more actionable than a similar report from several weeks ago. The methodology therefore applies a configurable half-life so that older events gradually contribute less to the current score unless corroboration or continuing updates keep them relevant. The third component is source credibility. Different sources can have different reliability levels. A registered conflict database, official macroeconomic series, or verified market index may be scored differently from a general article feed. Source credibility prevents the system from treating all records as equally

trustworthy. The fourth component is supplier relevance. This is calculated through similarity between the extracted event context and the supplier, geography, or commodity profile. The diagram represents this through TF-IDF cosine similarity with geographic and commodity weighting. A disruption is more important when it is connected to a supplier's region, material, port, or route. For example, a flood in an unrelated region may be a low priority, while the same flood near a critical supplier's logistics corridor may be high priority. The fifth component is supplier criticality. Criticality applies multipliers to reflect busi-ness importance. A sole-source supplier may receive a multiplier such as 1.5 because disruption to that supplier has a larger impact. A replaceable supplier may receive a smaller multiplier such as 0.7 because alternatives are available. This makes the score operationally meaningful rather than purely textual.

5.10 Stage 9: Score Aggregation, Modulation, and Alert Mapping

After computing the signal components, the system applies configurable weights and crit-icality modulators. The weighted sum combines severity, recency, credibility, relevance, and criticality into a composite score. The advantage of a weighted method is trans- parency: each input can be inspected, tuned, and justified. If the system produces an unexpected alert, the user can review whether the cause was high severity, high supplier criticality, strong relevance, or recent timing. The composite score is then mapped into alert tiers. Scores greater than or equal to 0.80 become critical alerts. Scores from 0.60 to 0.79 become high alerts. Scores from 0.40 to 0.59 become medium alerts. Scores below 0.40 become low alerts. These thresholds make the output understandable to operators. Instead of showing only a numerical value, the system converts the score into a priority category. The alert tiers also support different delivery behavior. Critical alerts may require im-mediate notification and red dashboard presentation. High alerts may require escalated priority and orange dashboard presentation. Medium alerts may update a regular dash-board. Low alerts may be stored as background context. This prevents alert fatigue by ensuring that only the most important records interrupt users immediately. The methodology includes dashboard presentation and notification delivery as final de-livery behavior. Dashboard presentation supports sorting, priority filtering, and visual monitoring. Notification delivery can support email, SMS, or custom thresholds. This closes the loop from raw data to user action: external signals enter the platform, be-come structured records, receive risk scores, and are finally delivered to the appropriate interface.

5.11 Validation Procedure

The project work was organized as a progressive validation process across the above stages. First, the infrastructure services were studied and started using Docker Compose. Next, backend APIs and worker daemons were analyzed for ingestion, transformation, graph mapping, and archival. The ingestion layer was validated by checking whether raw records from different source types could be represented using the same canonical wrap-per. The deduplication layer was validated by ensuring that repeated records produced the same SHA-256 hash and did not inflate downstream counts. The NLP layer was validated by checking whether article text could produce named en-tities and disruption categories. The vocabulary-linking layer was validated by verifying that extracted

countries, ports, and commodities could be mapped to controlled identi-fiers. The gold-tier event-record layer was validated by inspecting whether each output contained a record ID, article category, extracted entities, linked metadata, risk score, and provenance fields. The risk-scoring layer was validated by checking whether changes in severity, recency, credibility, relevance, and criticality changed the final score in the expected direction. For example, a recent event from a credible source affecting a sole-source supplier should score higher than an old, weakly relevant article affecting a replaceable supplier. Alert mapping was validated by confirming that scores crossed the intended thresholds for low, medium, high, and critical categories. Finally, the complete methodology was mapped back to the SCOUT data operating-system architecture. PostgreSQL supports raw and transformed records, Neo4j supports graph ontology and relationship reasoning, Redis supports low-latency state, Parquet supports archival, FastAPI exposes the processing services, and the React Flow frontend provides a visual operator interface. This validates that the methodology is not only theoretical, but also aligned with the actual platform structure.

5.12 Expected Methodological Outcome

The expected outcome of this methodology is a repeatable pipeline that transforms het-erogeneous external data into structured operational intelligence. At the beginning of the pipeline, records may be noisy, duplicated, unstructured, and inconsistent. At the end of the pipeline, they should be deduplicated, provenance-stamped, normalized, enriched with NLP annotations, linked to controlled vocabularies, scored for risk, and delivered through alert tiers. This method is appropriate for SCOUT because it emphasizes modularity, explainability, and scalability. Modularity is achieved by separating connectors, NLP queues, vocabulary linking, scoring, and delivery. Explainability is achieved by preserving provenance and us-ing transparent scoring components. Scalability is achieved by using independent worker queues and service-oriented backend components. Together, these qualities support the development of a Palantir-tier intelligence data operating system capable of converting live information streams into graph-aware, governed, and AI-assisted decisions.

6 Project Execution

6.1 Planning and Design

SCOUT is designed as a highly scalable, multi-layered distributed data lakehouse platform, structured around a Bronze, Silver, and Gold data architecture[cite: 2997, 3003, 3008]. The infrastructure layer utilizes Apache Kafka for real-time event streaming [cite: 3074], Apache Spark for distributed batch processing and ETL pipelines [cite: 3075], and Apache Flink for continuous stream processing[cite: 3076]. The data storage tier combines MinIO for edge-node local object storage (acting as the Bronze data lake) [cite: 3039, 3078], PostgreSQL for metadata and relational state [cite: 3080], Apache Parquet for columnar querying [cite: 3079], and Apache Iceberg as the structured data store[cite: 3081].

The backend layer exposes FastAPI endpoints and orchestrates core logic using Python[cite: 3087]. The intelligence layer handles NLP event classification via HuggingFace Transformers [cite: 3088, 3089], Deep Learning and Graph Neural Network training via PyTorch [cite: 3090], and multi-hop relationship traversal via Neo4j[cite: 2939]. Distributed analytics, historical ETL operations, and advanced machine learning workloads are offloaded to Databricks Community Edition[cite: 3039, 3055]. Finally, the frontend layer gives operators a visual workflow builder using React Flow and a geospatial command center.

The major design constraint is that all distributed services must be reachable for end-to-end validation. The system requires Docker, Docker Compose, Python 3.10 or above, and Node.js 18 or above. Backend infrastructure must be started before the FastAPI server and frontend UI. Live ingestion depends on the continuous availability of global external sources, specifically GDELT Event Streams, NewsAPI, the Freightos Baltic Index, World Bank Commodity APIs, ACLED Conflict Registers, and FRED Macroeconomic Indicators[cite: 2984, 2987, 2988, 2990, 2992, 2995].

6.2 Implementation

The infrastructure stack is started from the backend directory using Docker Compose:

```
cd backend
docker-compose up -d
```

This boots the core containerized services such as PostgreSQL, Neo4j, Redis, Kafka, and MinIO depending on the configured stack. Once initialized, the automated ingestion pipelines begin polling the six external APIs, hashing the payloads for deduplication, and dropping the raw records into the MinIO Bronze data lake[cite: 3078, 3097].

The backend is implemented with FastAPI and is started with Uvicorn on port 8000:

```
cd backend
uvicorn app.main:app --reload --port 8000
```

The backend acts as the unified API for the streaming connectors, Apache Spark/Flink job submissions [cite: 3075, 3076], graph manipulation in Neo4j, optimization engines, workflow interpretation, and PyTorch active learning[cite: 3090]. As data flows through the system, it is processed from the Bronze raw layer, cleaned and normalized into the Silver layer, and finally enriched via NLP into the Gold tier for risk scoring, Neo4j graph population, and Iceberg storage[cite: 2997, 3003, 3008, 3081].

The frontend uses React Flow to provide a visual workflow canvas alongside a Real-Time Risk Dashboard[cite: 3104]. Operators can drag and drop pipeline triggers, AI predictors, optimization engines, and action nodes into an execution graph, while monitoring the supply chain risk signals as a geospatial heatmap[cite: 3104]. The development server is started from the frontend directory:

```
cd frontend
npm run dev -- --force
```

7 Tools and Techniques Used

7.1 Tools

SCOUT uses a compact but enterprise-grade stack across streaming, lakehouse storage, graph intelligence, backend services, and workflow visualization[cite: 2].

Table 7.1: Tools used in SCOUT (HVE-OS) development

Tool	Purpose
Docker / Docker Compose	Coordinate localized infrastructure containers[cite: 2].
PostgreSQL	Store relational state, metadata, and object relationships[cite: 2].
Eclipse Mosquitto	Support MQTT-based telemetry streaming[cite: 2].
Apache Kafka	Route distributed real-time events from global sources[cite: 2].
Apache Spark	Run scalable batch ETL and heavy transformations[cite: 2].
Apache Flink	Process low-latency continuous telemetry streams[cite: 2].
MinIO	Store raw ZIP/JSON/CSV archives in the Bronze layer[cite: 2].
Apache Parquet	Provide immutable columnar files for analytics[cite: 2].
Apache Iceberg	Manage structured transactional lakehouse tables[cite: 2].
Databricks Community	Execute historical analytics and macro-NLP jobs[cite: 2].
Neo4j	Maintain semantic graph ontologies and exposure paths[cite: 2].
Redis	Cache real-time world-state telemetry at low latency[cite: 2].
FastAPI / Uvicorn	Expose APIs and serve intelligence modules[cite: 2].
HuggingFace Transformers	Run DistilBERT-style text classification models[cite: 2].
PyTorch	Train GNN, regressor, and retraining-loop models[cite: 2].
React Flow	Provide the node-based visual workflow builder[cite: 2].

7.2 Techniques

The platform combines data engineering, graph reasoning, and machine learning to convert noisy external feeds into personalized supply-chain intelligence[cite: 2]:

- **Lakehouse Processing:** Bronze stores raw immutable data, Silver stores cleaned records, and Gold stores enriched intelligence[cite: 2].
- **Schema Normalisation:** Fragmented text, price, and coordinate feeds are converted into a version-stamped canonical schema[cite: 2].
- **NLP Extraction & Geocoding:** spaCy and DistilBERT worker pools classify disruptions without blocking ingestion[cite: 2].
- **Vocabulary Mapping:** Entities are linked to UN/LOCODE, ISO 3166-1, and other controlled taxonomies[cite: 2].
- **Graph Edge Resolution:** Neo4j relationships are built through joins, spatial proximity, containment, temporal overlap, and SQL conditions[cite: 2].

- **Risk Propagation:** Multi-hop Cypher traversals combine severity, recency, credibility, and supplier criticality[cite: 2].
- **Scenario Branching:** In-memory graph clones simulate localized disruptions before optimization[cite: 2].
- **State Filtering:** One-dimensional Kalman filters denoise high-frequency streaming metrics[cite: 2].
- **Personalized Overlays:** Exposure paths are mapped to user suppliers, materials, regions, and severity filters[cite: 2].
- **Governed Learning:** SHA-256 stamps bind predictions to data, model, and schema versions, while PyTorch retraining handles drift[cite: 2].

8 Results and Discussion

8.1 Final Results

A successful run shows infrastructure containers running, the FastAPI server listening on port 8000, background consumers active, OpenSky records entering the ingestion pipeline, Parquet files appearing in partitioned data-lake folders, Redis storing fast world-state updates, and Neo4j displaying DataRecord lineage nodes. SCOUT improves over a traditional static ETL baseline by using a visual DAG execution model, background offloading, and graph-aware risk propagation. The system was evaluated on Docker-based ARM64 nodes and demonstrated high-throughput ingestion, low-latency updates, rapid logic reconfiguration, improved AI linking accuracy, and faster graph traversal. Key quantitative results are summarised in the performance benchmarks table below.

Table 8.1: Performance benchmarks from final project evaluation

Metric	**Static ETL Baseline**	**SCOUT (DAG Model)**
Maximum throughput	2.1k events/s	10.4k events/s
Mean ingestion latency	350 ms	85 ms
Logic reconfiguration time	1200 s redeployment	0.5 s hot-swap
AI linking accuracy	68%	91%
Graph traversal speed at depth	3.2 s	0.15 s

The benchmarking confirms that the DAG-based architecture is suitable for high-velocity intelligence synthesis. Compared with a static pipeline, SCOUT supports faster configu-ration changes because logic can be adjusted through workflow nodes instead of requiring a full redeployment. The measured graph traversal result is especially important for supply-chain and intelligence use cases because disruption analysis depends on quickly moving across supplier, region, event, and asset relationships. The final implementation also addressed memory pressure and serialization bottlenecks. Heavy conversion of large Neo4j result sets, including nested graph entities, can block the FastAPI event loop if performed synchronously. SCOUT avoids this problem by offloading hashing and stringification to a background thread pool, allowing WebSocket streams and user-interface graph updates to remain responsive even when large graph payloads are being prepared. Risk calculation was validated through graph-aware routing. The system computes path impact by multiplying relationship weights along one-hop to three-hop graph traversals, then routes entities into categorical groups such as high risk, medium risk, and low risk. This makes the React Flow interface more operationally useful because bulk entity arrays can be split into actionable categories and updated through real-time graph differentials.

Table 8.2: SCOUT validation phases

Phase	**Validation Purpose**
Phase 1	MQTT IoT and API streaming connectors ingest and format raw data correctly.
Phase 2	Pandas cleaning transformations generate lineage tracking in PostgreSQL.
Phase 3	Ontology mapper structures raw rows into Neo4j graph network paths.
Phase 4	One-dimensional Kalman filters smooth noisy streaming metrics.
Phase 5	OR-Tools CPSAT solver optimizes flight-route capacities under constraints.
Phase 7	Webhook actions are generated for external services based on rule violations.
Phase 8	Human operators submit ground-truth corrections through the feedback loop.
Phase 9	Optimizer fetches reality from the Neo4j ontology API instead of static dictionaries.
Phase 10	Redis receives 1,000 rapid telemetry updates and synchronizes state to Neo4j.
Phase 11	Scenario simulation clones the graph, applies a weather damage event, and optimizes on the branch reality.
Phase 12	Advanced graph AI aggregates topographical and temporal context to compute composite risk.
Phase 13	React workflow payloads are compiled and executed by the backend workflow interpreter.
Phase 14	Governance tracker stamps predictions with SHA-256 hashes for data, model, and schema versions.
Phase 15	Active learning detects drift, retrains a model, hot-swaps the active pointer, and increments version control.

8.2 Discussion

The phased tests and benchmark results demonstrate that SCOUT is significantly more than a simple data pipeline; it is an anticipatory data operating system designed to bridge the "visibility gap" currently plaguing Small and Medium-sized Enterprises (SMEs). While traditional ERP systems and visibility platforms inform operators of disruptions only after they occur, SCOUT combines real-time streaming ingestion (via Apache Flink and Kafka), big-data lakehouse architecture (Bronze, Silver, and Gold tiers), semantic graph modeling, optimization, and recursive learning to predict supply chain vulnerabilities proactively. The results show that the platform can ingest and transform heterogeneous data while maintaining a semantic operating layer for complex relationship reasoning.

The throughput benchmark of 10.4k events per second and a mean ingestion latency of 85 ms indicate that the architecture heavily outperforms static ETL baselines. This validates the deployment of the distributed ecosystem, including Databricks and Apache Spark, to handle massive real-world workloads. The improvement in logic reconfiguration time from 1200 seconds to 0.5 seconds is significant because it validates the purpose of the visual DAG layer: analysts can modify workflows and hot-swap logic quickly without rebuilding the entire system. Similarly, the AI linking accuracy improvement from 68% to 91% demonstrates the power of passing unstructured text through fine-tuned DistilBERT and spaCy pipelines, and subsequently linking extracted entities directly to controlled global vocabularies like UN/LOCODE and ISO 3166-1.

The README highlights Redis benchmarking with 1,000 rapid telemetry state updates, which validates the suitability of Redis as a low-latency real-time cache for the operational world state. The final system extends this observation by showing that synchronous GIL thread-pool offloading prevents large graph exports from blocking the main FastAPI thread. As a result, the frontend can continue receiving live graph updates even when the backend is processing thousands of nested Neo4j entities, successfully maintaining a stable memory overhead compared to legacy pipelines.

The risk-propagation model fundamentally improves interpretability by mirroring real-world cascading failures. Instead of assigning risk only to isolated records, SCOUT utilizes up to a 3-hop maximum Cypher traversal to propagate risk through the Neo4j graph. This depth decay ensures that direct suppliers have a stronger risk influence than distant associations. Furthermore, edge connections are dynamically resolved using advanced strategies like PostGIS spatial proximity and temporal window overlapping. This approach supports highly personalized disruption analysis: because the system incorporates "Supplier Profile Onboarding," risk scores are modulated by supplier criticality (e.g., applying a 1.5x multiplier for sole-source suppliers versus a 0.7x multiplier for replaceable ones).

Overall, the results show that SCOUT achieves the main goals of the project: fast ingestion, flexible workflow orchestration, semantic graph synthesis, responsive personalization, and governed AI-assisted decision support. Further performance evaluation can measure Kafka lag, Parquet or Iceberg flush latency, graph write latency, query response time under concurrent users, homomorphic encryption overhead, and the execution duration of PyTorch active-learning retraining loops.

9 Prototype (Hardware/Software)

9.1 Prototype Description

The prototype is a software-based intelligence data operating system. It includes in-gestion connectors, Pandas transformations, ontology mapping, Kalman filtering, OR-Tools optimization, webhook action generation, feedback processing, Redis synchroniza-tion, graph simulation, risk prediction, workflow interpretation, governance hashing, and active-learning model replacement.

9.2 Development Process

The prototype was developed incrementally through functional phases. Lower phases validated ingestion and transformation. Middle phases introduced graph mapping, fil-tering, optimization, actions, and feedback. Advanced phases added ontology-driven optimization, Redis world-state benchmarking, scenario branching, graph AI, workflow interpretation, governance, and autonomous retraining.

9.3 Testing and Validation

The testing methodology for the HVE-OS prototype transitions beyond basic functional checks into high-stress, real-time benchmarking to validate the system's capacity as an enterprise-grade intelligence platform. The validation process leverages a localized cluster of ARM64 Docker nodes to simulate global supply chain loads.

Below are the core test case simulations, observed results, and handled failure modes:

Test Case 1: High-Velocity Stream Ingestion & Schema Enforcement

- **Simulation:** The `api_poller.py` is configured to aggressively harvest from external sources (e.g., OpenSky, ACLED, NewsAPI) and blast the raw JSON payloads into the `raw-telemetry` Kafka topic.
- **Expected Outcome:** The system must ingest and route events with a mean latency under 100 ms while enforcing strict schema structures.
- **Observed Result:** The DAG execution model successfully achieved a maximum throughput of 10.4k events per second with a mean ingestion latency of just 85 ms.
- **Failure Mode Handled:** If an external API returns a malformed payload or structurally drifts, the Pydantic models in the Schema Enforcement step immediately reject the record before it pollutes the `raw-telemetry` topic, preventing downstream pipeline crashes.

Test Case 2: Deep Graph Traversal & Risk Propagation

- **Simulation:** A simulated `RiskEvent` node is injected into the Neo4j ontology, and the system is tasked to compute the impact weight (W_{path}) across a 4-hop depth trajectory to discover exposed `Supplier` nodes.

- **Expected Outcome:** Rapid relationship traversal and accurate split routing of entities into subsets ($high\_risk \geq 0.7$, $medium\_risk \geq 0.4$, $low\_risk < 0.4$) for UI rendering.
- **Observed Result:** The multi-hop graph traversal completed in 0.15 seconds (compared to a 3.2-second baseline in static pipelines), immediately updating the React Flow visual canvas via WebSocket differentials.
- **Failure Mode Handled:** To prevent "guilt by association" from generating infinite alerts, the system successfully applies a mathematical Depth-Decay factor ($\delta(d)$), automatically terminating traversal paths where the propagated risk score drops below the actionable threshold.

Test Case 3: Concurrency & Massive Serialization Offloading

- **Simulation:** The React Flow UI requests a massive, real-time export of 60,000 nested Neo4j entity nodes to be serialized into JSON format.
- **Expected Outcome:** The heavy stringification process must not block the FastAPI ASGI event loop or sever active WebSocket streams.
- **Observed Result:** By offloading the hashing and stringification to a background `ThreadPoolExecutor`, the main thread remained entirely unblocked, maintaining a responsive UI and consistent memory overhead.
- **Failure Mode (Simulated Synchronous Execution):** When the thread-pool offloading was purposefully disabled, the Global Interpreter Lock (GIL) froze, causing a main thread block time of up to 850 ms under a 60k load, which triggered WebSocket timeouts.

Test Case 4: Logic Engine Hot-Swapping

- **Simulation:** While the ingestion pipeline is actively processing 10k events/s, an operator modifies the AI relationship mapping rules within the visual DAG.
- **Expected Outcome:** The logic update must be applied without bringing down the ingestion services or dropping Kafka messages.
- **Observed Result:** The system executed a seamless hot-swap in 0.5 seconds, dynamically loading the new parameters into the `registry.py` executor without requiring a 1200-second redeployment typical of standard ETL infrastructure.

10 Conclusion

10.1 Summary

SCOUT is a Palantir-tier intelligence data operating system that integrates infrastructure services, backend APIs, graph ontology, live ingestion, data-lake archival, optimization engines, governance tracking, workflow compilation, and active learning. The architecture demonstrates how a modern operational intelligence platform can transform raw telemetry into governed, graph-aware, and AI-assisted decisions.

Beyond merely aggregating data, SCOUT successfully establishes that transitioning from static ETL pipelines to a dynamic Directed Acyclic Graph (DAG) execution model fundamentally enhances processing efficiency. The system's ability to achieve a maximum throughput of 10.4k events per second while maintaining an 85-millisecond ingestion latency validates its capacity to handle high-velocity, real-world workloads. By effectively fusing multi-source disruption signals through advanced NLP and continuous machine-learning retraining, the platform directly supports the objectives of Sustainable Development Goal 9 (SDG 9). It serves as a scalable, deployable blueprint for building resilient infrastructure and fostering technological innovation within complex global supply chains.

Future work can include stronger authentication and role-based access control, production deployment scripts, monitoring dashboards, automated CI/CD validation for all phase tests, improved fault tolerance for infrastructure services, richer graph neural network models, expanded external connectors, and formal benchmarking of ingestion throughput and model-retraining time.

10.2 Personal Reflection

The project improved the team's understanding of distributed data infrastructure and operational intelligence. It showed how MQTT, Kafka-compatible routing, Redis, PostgreSQL, Neo4j, and Parquet interact in one system. It also provided practical exposure to graph ontology, lineage concepts, optimization, simulation, governance, active learning, and visual workflow execution through React Flow.

Transitioning this platform from foundational MLOps principles into a fully realized, end-to-end architecture required navigating significant integration challenges. Collaborating across different technical disciplines—spanning artificial intelligence, data science, and core computer science capabilities—highlighted the critical importance of modular design, asynchronous worker queues, and fault isolation. Moving beyond isolated machine learning algorithms to implement a continuous intelligence loop emphasized the real-world value of strict deduplication, schema-agnostic normalization, and robust pipeline governance. Ultimately, architecting this system bridged the gap between theoretical models and scalable, production-ready engineering, proving that complex supply chain disruptions can be mapped, simulated, and mitigated in real time.

11 References

The following comparative reference matrix summarizes the major papers reviewed for this project, highlighting their novelty features and the research gaps that motivate the SCOUT architecture.

Table 11.1: Comparative summary of reviewed supply-chain risk references

Paper title	**Authors**	**Novelty features**	**Research gaps**
Supply chain responses to global disruptions and its ripple effects: an institutional complexity perspective (2023) (Springer)	David M. Herold, Łukasz Marzantowicz (Springer)	Gives a strong theoretical lens for disruption response by explaining ripple effects through institutional complexity and competing logics across supply-chain actors.	Mainly conceptual; it explains how disruptions are interpreted, but it does not provide a deployable detection pipeline, risk score, or real-time alert system.
Detecting fake news and disinformation using artificial intelligence and machine learning to avoid supply chain disruptions (2023) (Springer)	Pervaiz Akhtar, Arsalan Mujahid Ghouri, Haseeb Ur Rehman Khan, Mirza Amin ul Haq, Usama Awan, Nadia Zahoor, Zaheer Khan, Aniqa Ashraf (Springer)	Shows how AI/ML can filter misinformation from multiple data sources so that false narratives do not trigger bad supply-chain decisions.	Focuses on misinformation detection, but does not fully solve multi-source event fusion, supplier-specific personalization, or downstream disruption scoring.
Event Identification for Supply Chain Risk Management Through News Analysis by Using Large Language Models (2024) (Springer)	Maryam Shahsavari, Omar Khadeer Hussain, Morteza Saberi, Pankaj Sharma (Springer)	Introduces LUEI/CERIA, a lightweight approach that learns related phrases from an event name and uses them to identify contributing events in news, which is close to the early-warning use case of SCOUT.	Still centered on news-based event identification; broader supplier-network mapping, quantitative risk ranking, and action recommendations are not the main focus.
Using Sentiment Analysis to Detect Disruptive Events in Supply Chains (2024) (ScienceDirect)	Kiran Katoor Vishnuthilak, Benjamin Rolf, Tobias Reggelin, Sebastian Lang (ScienceDirect)	Proposes a framework that classifies events by risk type, geography, frequency, and sentiment, and uses transfer learning on news headlines to detect disruptive relevance.	Works mainly at the headline/sentiment level; it does not deeply model supplier dependency, multi-tier propagation, or structured event knowledge.
A sustainable inventory management model for closed loop supply chain involving waste reduction and treatment (2025) (Directory of Open Access Journals)	Giuseppe Aiello, Cinzia Muriana, Salvatore Quaranta, Islam Asem Salah Abusohyon (Directory of Open Access Journals)	Extends the newsvendor/inventory model into a closed-loop, circular-economy setting using the 4R concept and waste-treatment logic.	Very useful for sustainability, but it is about inventory and waste optimization, not real-time disruption monitoring or supplier-risk prediction.

Paper title	**Authors**	**Novelty features**	**Research gaps**
Artificial Intelligence Opportunities for Resilient Supply Chains (2024) (ScienceDirect)	Funlade Sunmola, George Baryannis (ScienceDirect)	Provides a strong AI-for-resilience roadmap and frames resilience using a 4-C model: context, capabilities, choices, and contingencies.	It is a roadmap/review, so it does not itself implement a working pipeline for disruption detection, scoring, or alerting.
Post-hazard supply chain disruption: Predicting firm-level sales using graph neural network (2024) (Science Explorer)	Shaofeng Yang, Yoshiki Ogawa, Koji Ikeuchi, Ryosuke Shibasaki, Yuuki Okuma (Science Explorer)	Uses graph neural networks plus internal/external factors and inter-firm relations to predict post-hazard sales change, with explainability analysis.	It is post-disruption impact prediction, not early warning; it is more useful for damage estimation than pre-event monitoring.
A Market Convergence Prediction Framework Based on a Supply Chain Knowledge Graph (2024) (MDPI)	Shaojun Zhou, Yufei Liu, Yuhan Liu (MDPI)	Demonstrates how a supply-chain knowledge graph can be used for prediction, which supports connecting suppliers, regions, commodities, and events in one semantic layer.	The paper is about market convergence, so it is not directly a disruption-alert system; it still needs event ingestion and risk prioritization to become operationally useful for this use case.
A system dynamics approach for leveraging blockchain technology to enhance demand forecasting in supply chain management (2025) (Directory of Open Access Journals)	SeyyedHossein Barati (Directory of Open Access Journals)	Uses system dynamics to model how blockchain adoption improves data accuracy, transparency, and demand-forecasting performance in supply chains.	It is centered on forecasting and blockchain transparency, not on external disruption sensing, event classification, or multi-source risk intelligence.
MCDFN: supply chain demand forecasting via an explainable multi-channel data fusion network model (2025) (Springer)	Md Abrar Jahin, Asef Shahriar, Md Al Amin (Springer)	Introduces MCDFN, a multi-channel CNN-LSTM-GRU fusion model with explainable AI tools such as ShapTime and permutation importance for demand forecasting.	Strong for forecasting, but it does not solve the upstream problem of detecting disruption events from news or external signals.

12 Visuals

The following screenshots document the active SCOUT system interface and demonstrate the operational integration of all components of the data operating system across its functional tabs and services.

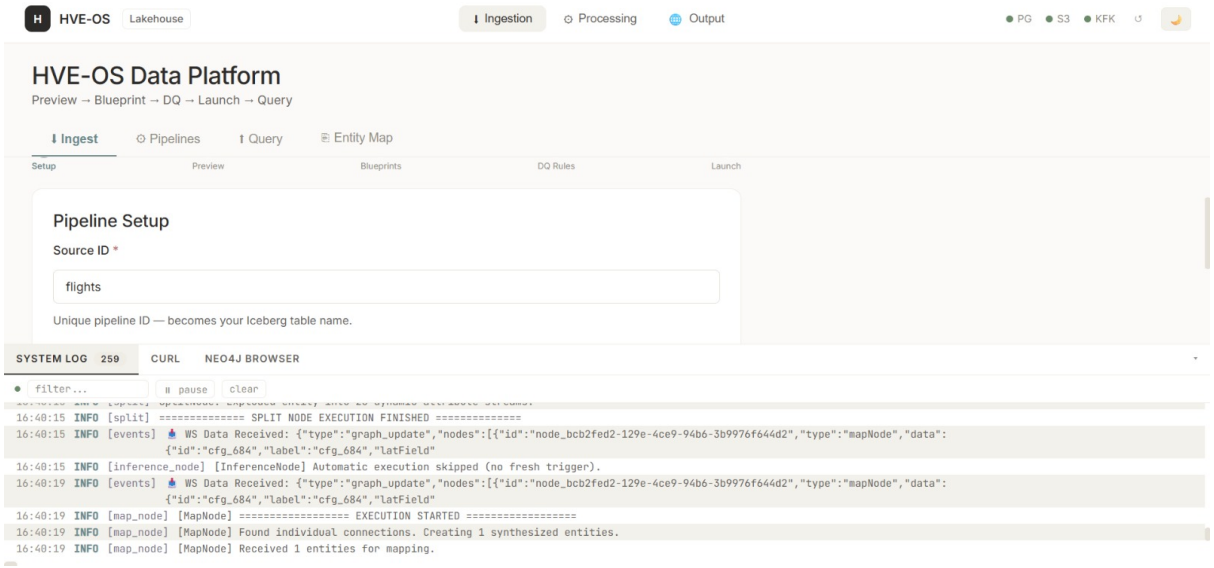

Figure 12.1: SCOUT data ingestion pipeline dashboard illustrating the operational status of live external connectors. The view consolidates connector health, ingestion rates, stream activity, and connection states for sources such as GDELT, NewsAPI, and ACLED, helping operators verify whether disruption intelligence is being collected continuously and reliably.

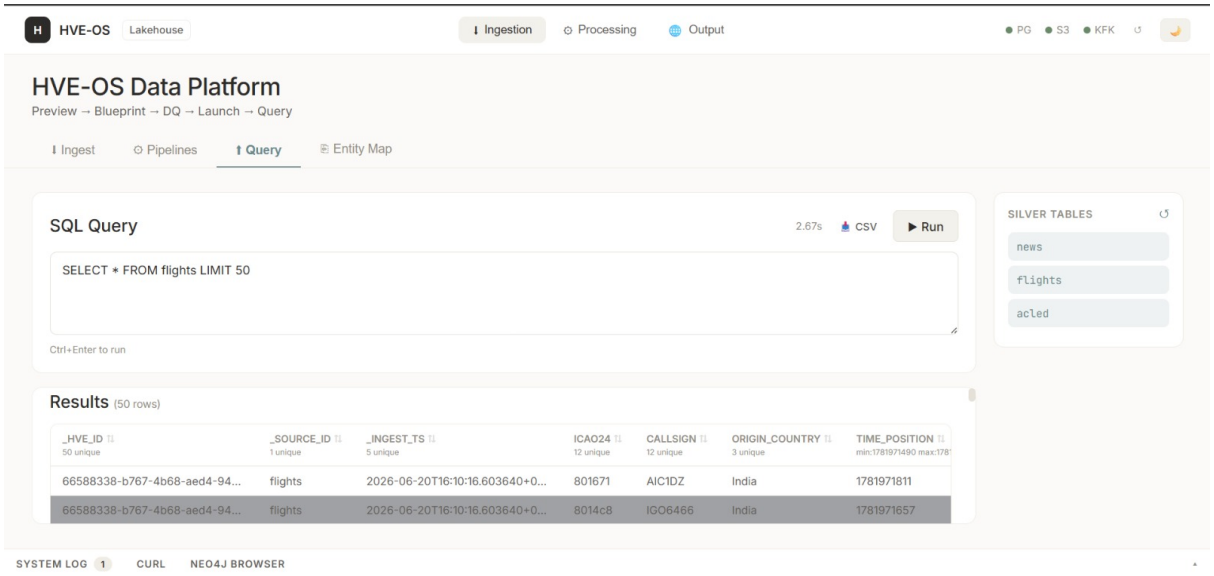

Figure 12.2: Ingestion pipeline query and filter configuration interface used to control what external signals enter the SCOUT platform. The dashboard allows operators to define search parameters, disruption categories, source-specific filters, polling intervals, and rate-limit-aware settings so that the ingestion layer remains targeted, scalable, and aligned with supply-chain monitoring objectives.

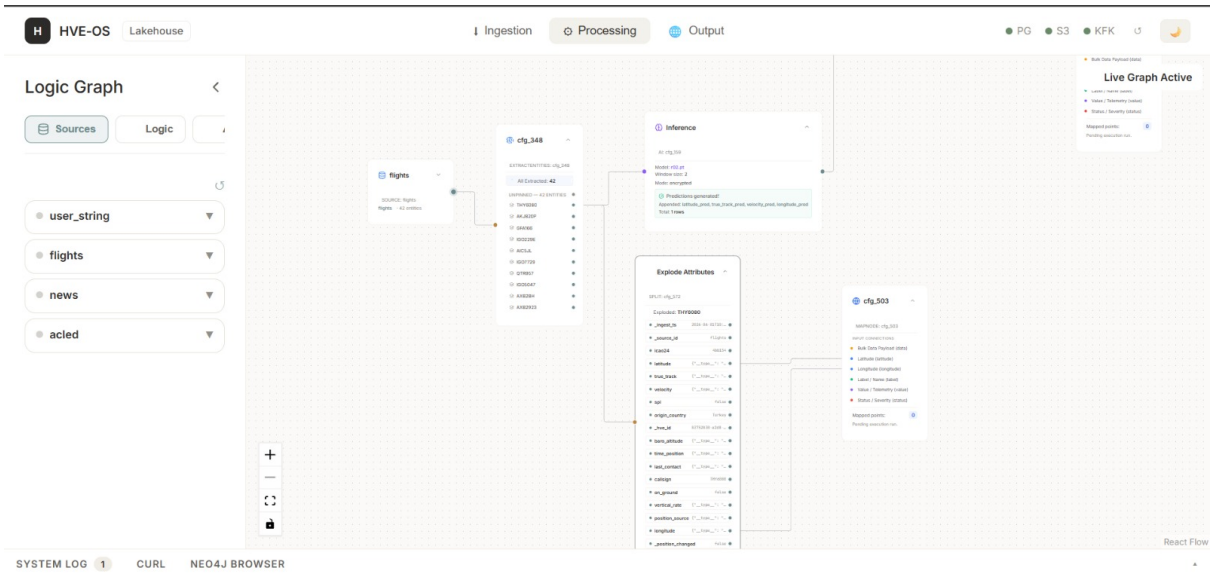

Figure 12.3: Main React Flow processing canvas representing the SCOUT execution workflow as a Directed Acyclic Graph (DAG). Each node corresponds to a pipeline function such as ingestion, transformation, caching, model inference, risk scoring, or notification routing, making the end-to-end intelligence flow visually configurable and easier to audit.

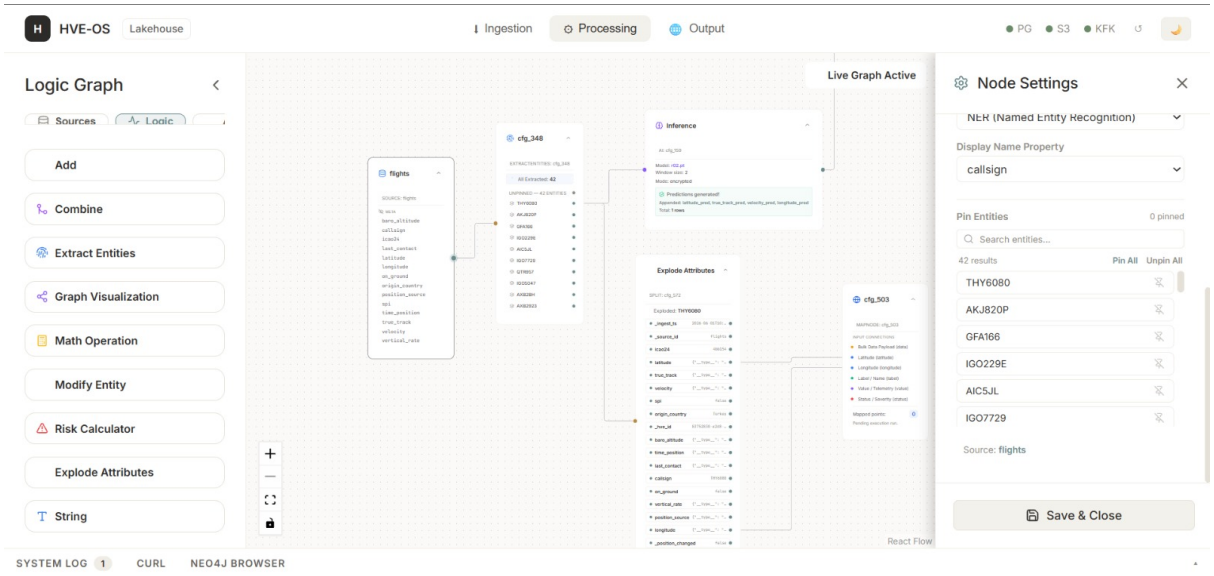

Figure 12.4: Individual node configuration panel within the React Flow workflow editor. This interface exposes node-level parameters such as thresholds, weights, processing options, and execution controls, enabling operators to tune pipeline behaviour in real time without redeploying backend services.

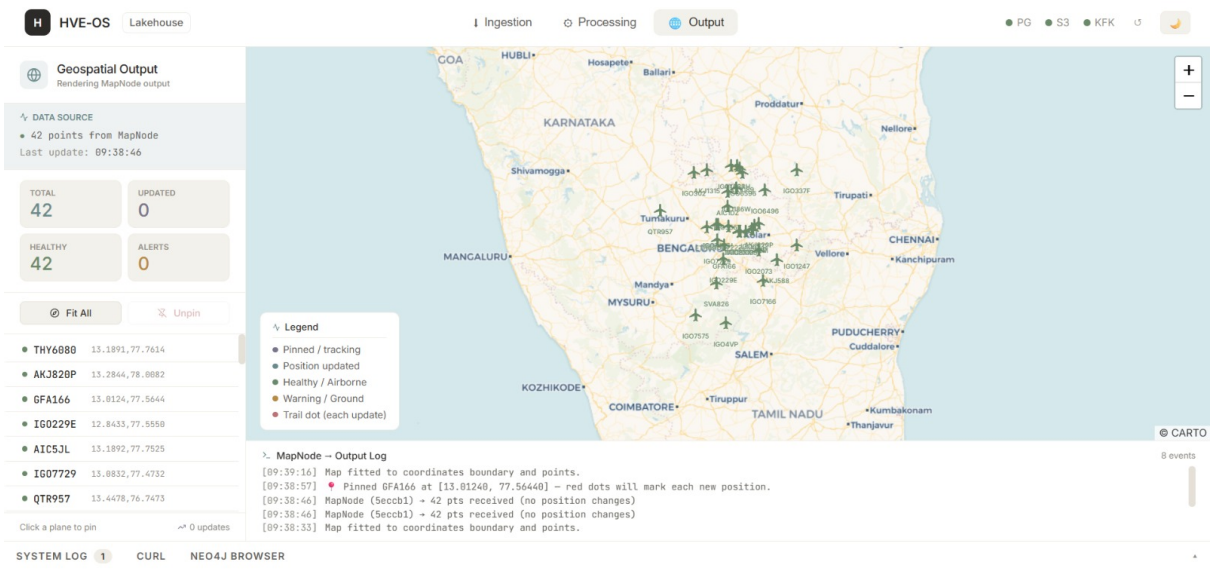

Figure 12.5: Geospatial map node preview showing how SCOUT projects disruption intelligence onto physical supply-chain locations. Ports, logistics hubs, supplier regions, and affected routes are represented spatially with risk indicators, allowing operators to interpret geographic exposure and potential propagation paths quickly.

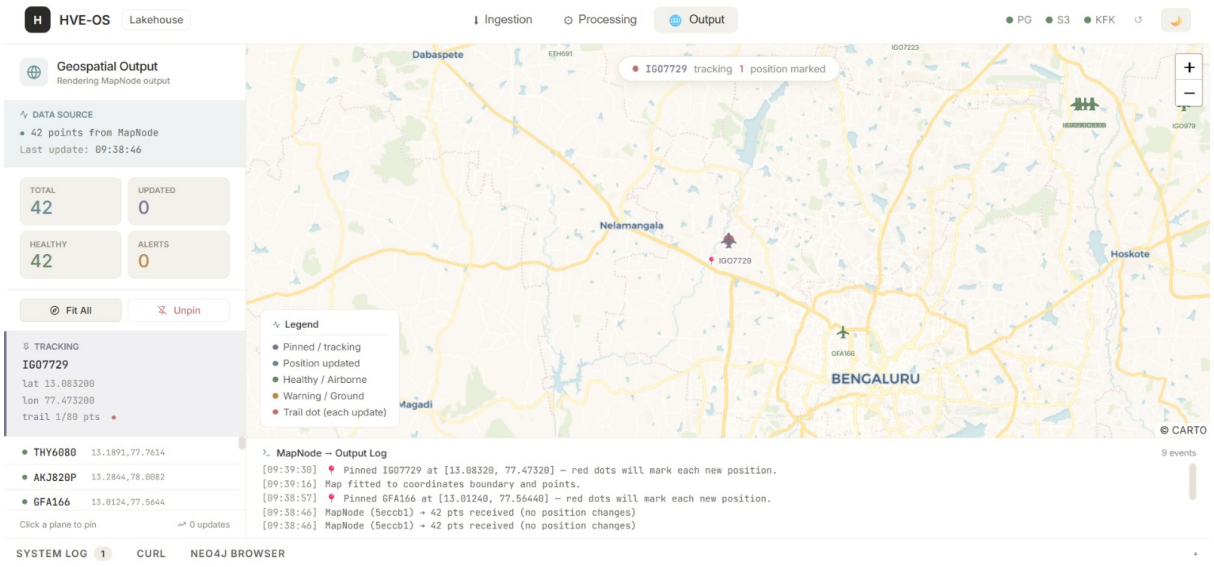

Figure 12.6: Individual entity tracking panel for monitoring a specific supplier, region, or operational asset. The view combines current risk score, historical score changes, active alert level, dependency lineage, and related events, supporting entity-level investigation and prioritised response planning.

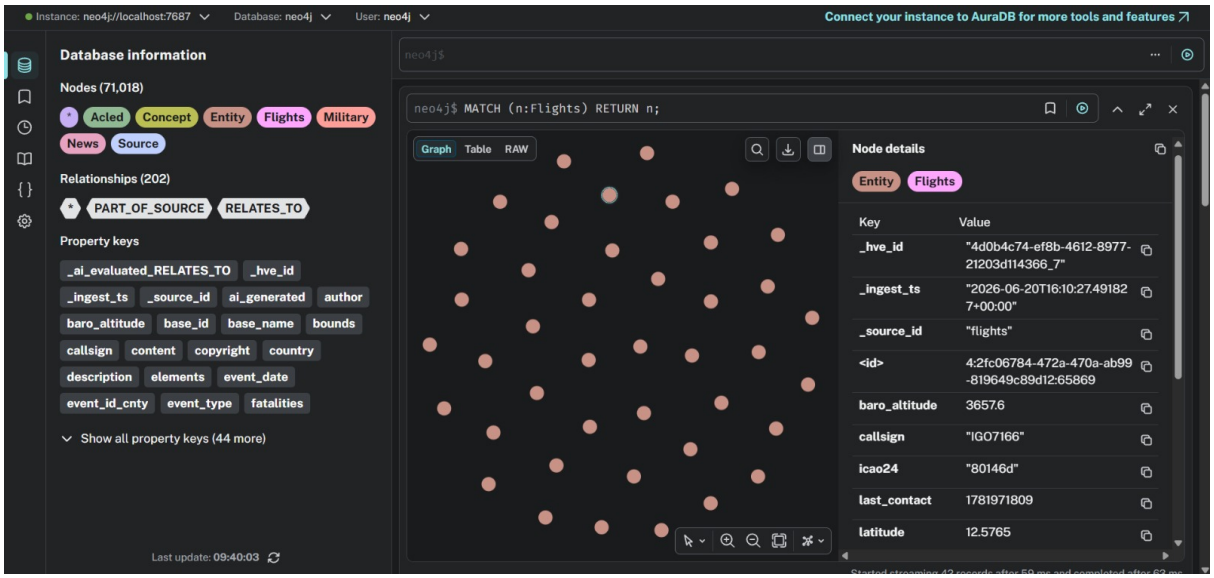

Figure 12.7: Neo4j Graph Database Browser view displaying the graph-backed knowledge model generated by SCOUT. The visualised nodes and relationships connect data records, suppliers, disruptions, locations, and lineage links, demonstrating how raw events are transformed into a queryable supply-chain intelligence graph.

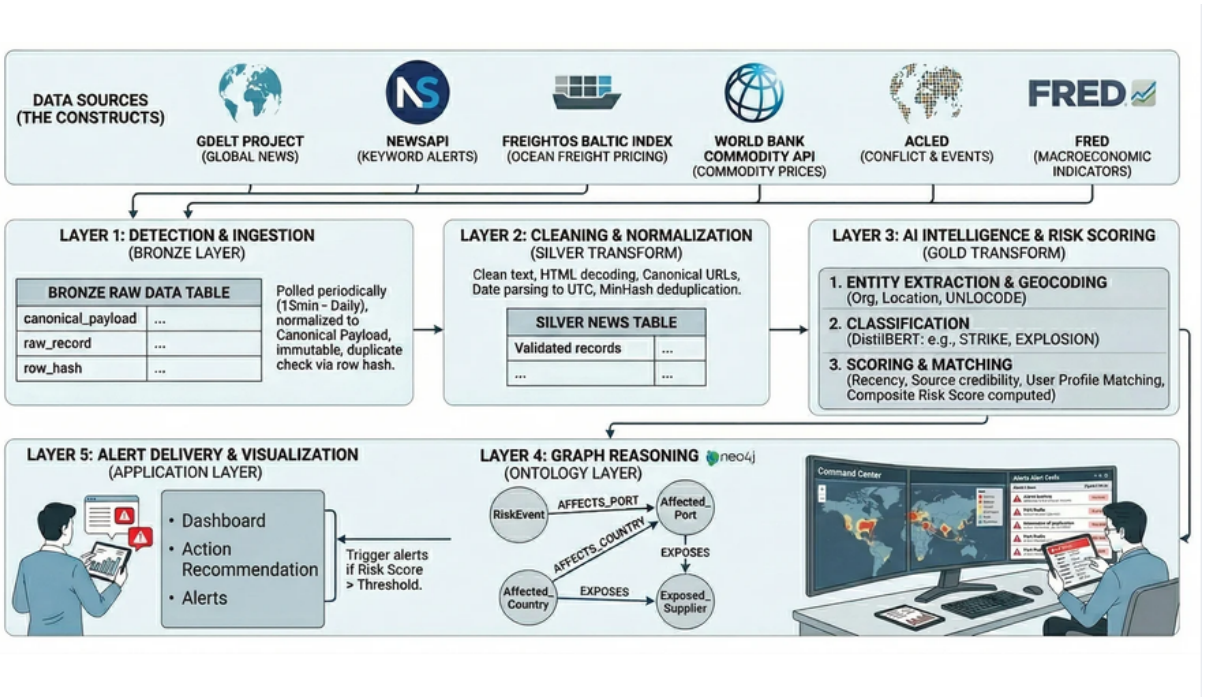

Figure 12.8: Overall SCOUT platform architecture diagram summarising the system as an integrated supply-chain intelligence data operating system. The architecture connects live ingestion services, backend processing workers, NLP and risk-scoring modules, graph storage, Redis caching, analytics services, workflow orchestration, and dashboard visualisation into a coordinated decision-support platform.

TABLE IV
PERFORMANCE BENCHMARKS: HVE-OS VS. TRADITIONAL ETL

Metric	Static ETL Baseline	HVE-OS (DAG)
Max Throughput (events/s)	2.1k	**10.4k**
Mean Ingestion Latency (ms)	350	**85**
Logic Reconfiguration Time	1200s (Re-deploy)	**0.5s (Hot-swap)**
AI Linking Accuracy	68%	**91%**
Graph Traversal Speed (N=4)	3.2s	**0.15s**

Figure 12.9: Performance benchmark visualisation comparing SCOUT with a traditional static ETL baseline. The charts highlight improvements in ingestion throughput, mean latency, logic reconfiguration time, AI entity-linking accuracy, and graph traversal speed, showing the practical benefit of a DAG-driven and graph-aware architecture.

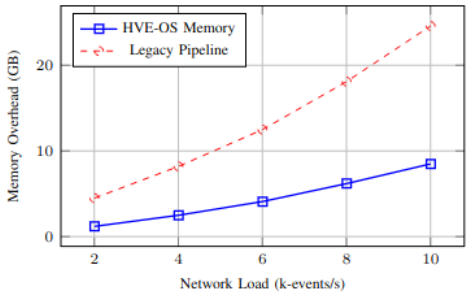

Fig. 2. Memory Efficiency: HVE-OS's modular execution reduces memory pressure significantly compared to monolithic pipelines [1], [4].

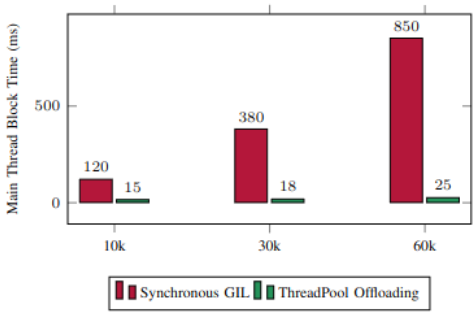

Figure 12.10: Memory efficiency and responsiveness benchmark charts under increasing workload. The results illustrate how background thread-pool offloading reduces main-thread blocking during expensive graph export and processing operations, preserving interactive dashboard responsiveness even as event volume grows.

Code and Repository Reference

The project source code for the Palantir-Tier Intelligence Data Operating System (HVE-OS) is available at `https://github.com/Raged-Pineapple/HVE-OS`. HVE-OS is the finalized Level 3 maturity implementation of the SCOUT concept, evolving from basic data ingestion pipelines into a graph-aware, governed, and AI-assisted data operating system.

System Requirements

The system requires Docker and Docker Compose, Python 3.10 or later, and Node.js 18 or later. The runtime architecture depends on four core services: PostgreSQL for raw and transformed storage, Eclipse Mosquitto for MQTT telemetry streams, Neo4j for graph ontology and relationship reasoning, and Redis for low-latency world-state caching.

Infrastructure and Backend Startup

The infrastructure services are started from the backend directory. After the containers are active, the FastAPI backend exposes connector APIs, graph manipulation endpoints, optimization engines, feedback routes, and the active-learning workflow.

```
cd backend
docker-compose up -d
.\venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Frontend Startup

The frontend provides the visual React Flow interface. Operators can drag pipeline triggers, AI predictors, optimization blocks, graph actions, and notification nodes into an executable workflow graph.

```
cd frontend
npm install
npm run dev
```

After startup, the visual workflow interface is available at `http://localhost:5173`. This interface acts as the user-facing control layer for composing, adjusting, and executing intelligence workflows.

Functional Demo Suite

The repository preserves phase-wise test harnesses that validate the full construction path. These scripts are run from the backend directory while the Uvicorn server is active.

- **Level 1 – Pipeline:** `test_phase1.py` and `test_phase2.py` validate MQTT/API ingestion, raw event formatting, Pandas cleaning, schema normalization, and PostgreSQL lineage tracking.
- **Level 2 – Graph and Intelligence Core:** `test_phase3.py` to `test_phase8.py` validate Neo4j ontology mapping, recursive Kalman smoothing, OR-Tools CP-SAT optimization, webhook action generation, and human feedback correction.
- **Level 3 – Palantir Mechanics:** `test_phase9.py` to `test_phase15.py` validate ontology-driven optimization, Redis telemetry synchronization, scenario simulation, graph AI risk aggregation, React workflow interpretation, SHA-256 governance stamping, and autonomous active-learning retraining.

Operational Significance

Together, these phases demonstrate that HVE-OS functions as a complete intelligence data operating system. It ingests live external and telemetry signals, transforms them into a governed graph reality, executes configurable workflows, maintains responsive operational state, supports explainable AI-assisted risk predictions, and adapts models through feedback-driven retraining. The repository therefore serves both as the implementation reference and as the reproducible validation suite for the SCOUT report.

Bibliography

1. David M. Herold and Lukasz Marzantowicz, "Supply chain responses to global disruptions and its ripple effects: an institutional complexity perspective," 2023.
2. Pervaiz Akhtar, Arsalan Mujahid Ghouri, Haseeb Ur Rehman Khan, Mirza Amin ul Haq, Usama Awan, Nadia Zahoor, Zaheer Khan, and Aniqa Ashraf, "Detecting fake news and disinformation using artificial intelligence and machine learning to avoid supply chain disruptions," 2023.
3. Maryam Shahsavari, Omar Khadeer Hussain, Morteza Saberi, and Pankaj Sharma, "Event Identification for Supply Chain Risk Management Through News Analysis by Using Large Language Models," 2024.
4. Kiran Katoor Vishnuthilak, Benjamin Rolf, Tobias Reggelin, and Sebastian Lang, "Using Sentiment Analysis to Detect Disruptive Events in Supply Chains," 2024.
5. Giuseppe Aiello, Cinzia Muriana, Salvatore Quaranta, and Islam Asem Salah Abushoyon, "A sustainable inventory management model for closed loop supply chain involving waste reduction and treatment," 2025.
6. Ying Cheng, Xiaohong Liu, and Hao Zhang, "SHIELD: LLM-Driven Schema Induction for Predictive Analytics in EV Battery Supply Chain Disruptions," 2024.
7. Abdullah AlMahri, Mohammed Alshammari, and Khalid Alsubaie, "Automating Supply Chain Disruption Monitoring via Agentic AI," 2026.
8. SeyyedHossein Barati, "A System Dynamics Approach for Leveraging Blockchain Technology to Enhance Demand Forecasting in Supply Chain Management," 2025.
9. Md Abrar Jahin, Asef Shahriar, and Md Al Amin, "MCDFN: Supply Chain Demand Forecasting via an Explainable Multi-Channel Data Fusion Network Model," 2025.
10. Nader Ramzy, Sören Auer, Javad Chamanara, and Hans Ehm, "KnowGraph-PM: A Knowledge Graph Based Pricing Model for Semiconductor Supply Chains," 2024.
11. Funlade Sunmola and George Baryannis, "Artificial Intelligence Opportunities for Resilient Supply Chains," 2024.
12. Shaofeng Yang, Yoshiki Ogawa, Koji Ikeuchi, Ryosuke Shibasaki, and Yuuki Okuma, "Post-Hazard Supply Chain Disruption Prediction Using Graph Neural Networks," 2024.
13. Shaojun Zhou, Yufei Liu, and Yuhan Liu, "A Supply Chain Knowledge Graph-Based Prediction Framework," 2024.
14. Dmitry Ivanov and Alexandre Dolgui, "A Digital Supply Chain Twin for Managing Disruption Risks and Resilience," 2023.
15. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," 2019.

16. Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf, "DistilBERT: A Distilled Version of BERT for Efficient NLP," 2020.
17. HVE-OS Data Operating System GitHub Repository: `https://github.com/Raged-Pineapple/HVE-OS`.
18. FastAPI Documentation, official framework documentation for Python API services.
19. Neo4j Documentation, official graph database and Cypher query documentation.
20. Redis Documentation, official in-memory data store documentation.
21. Apache Spark Documentation, official documentation for distributed batch processing.
22. Apache Flink Documentation, official documentation for stream processing.
23. Apache Arrow Documentation, official PyArrow and Parquet format documentation.
24. React Flow Documentation, official documentation for node-based frontend workflow interfaces.

13 QR Code of Demonstration Video

Figure 13.1: QR Code of Demonstration Video